Hafta 1 — Derste Yapılacak Uygulamalar

> **Ortam:** Sınıfta öğrenci bilgisayarı **yok**. Tüm uygulamalar **hoca ekranında canlı** yapılır; öğrenci tahmin eder, hoca çalıştırıp doğrular. Kodlama öğrenciye **ev görevi** olarak devredilir. **Toplam süre:** 36 dakika — blok sürelerinin **içindedir**, ek süre değildir (bkz. `H01-anlatim.md` §2) **Gerekli:** Projeksiyon, terminal, .NET 10 SDK, bir tarayıcı. İnternetsiz yedek planlar her uygulamanın sonunda.

🔧 Hazırlık — Derse girmeden önce (5 dk, ders dışı)

- [ ] `~/ders/IlkProje` altında `dotnet restore` **edilmiş, çalışır bir proje** hazır dursun (ağ giderse buradan devam edilir — bkz. her uygulamanın "Yedek plan"ı)
- [ ] `~/ders/bos` diye **boş bir klasör** hazır olsun (Uygulama B burada sıfırdan başlar)
- [ ] Terminal yazı tipi **en az 18 punto**; tema açık renk (projeksiyonda koyu tema okunmaz)
- [ ] Tarayıcıda yalnızca bir sekme açık olsun; yer imleri çubuğu kapalı
- [ ] `dotnet --version` bir kez çalıştırılıp **10.x** döndüğü doğrulansın

Uygulama A — Bir İsteğin Ham Hâlini Okumak

Süre: 10 dakika · **Blok:** A (HTTP sözleşmesi bölümünden hemen sonra) **Amaç:** Ders notundaki "istek dört bölümden oluşur" iddiasının gerçek olduğunu göstermek; öğrencinin `Web Teknolojileri`'nden bildiği DevTools'u **sunucu gözüyle** yeniden okutmak.

Adım 1 — Ağ sekmesini hazırla

```
Tarayıcı → F12 → Network sekmesi
"Preserve log" işaretle
```

> **Sınıfa sor:** *"Bu paneli geçen dönem gördünüz. O zaman neye bakıyordunuz?"* Beklenen cevap: yüklenme süresi, dosya boyutu. **Vurgula:** "Bu dönem aynı panele **yanıtı üreten taraf** olarak bakacağız."

Adım 2 — Bir sayfa aç ve ilk isteği seç

```
Adres çubuğu → www.sakarya.edu.tr → Enter
Listenin en üstündeki "document" satırına tıkla → Headers
```

Beklenen çıktı: `Request Headers` ve `Response Headers` bölümleri.

> **Sınıfa sor (tahmin ettir, sonra göster):**
>
> 1. "*Bu istek hangi metotla gitti?*" → `GET`
> 2. "*Yanıtın durum kodu kaç?*" → `200`
> 3. "*Sunucu bize hangi tipte veri döndüğünü nasıl söylüyor?*" → `Content-Type` başlığı

⚠️ **Sık hata:** Paneli açmadan sayfayı yüklemek — o zaman istekler kaydedilmez. Önce panel, sonra sayfa.

Adım 3 — Ham hâli göster

Chrome/Edge'de `Headers` sekmesinin sağ üstünde **"Raw"** düğmesi vardır. Tıkla.

Vurgulanacak nokta: *"Gördüğünüz bu düz metin, ağdan geçen şeyin ta kendisi. HTTP bir metin protokolüdür. Bu dönem bu metni **siz üreteceksiniz.**"*

Adım 4 — Stateless olmayı göster (bu adımı atlamayın)

```
Aynı sayfayı yenile (F5)
Yeni gelen "document" isteğini seç → Request Headers
```

> **Sınıfa sor:** "*Sunucu bu ikinci isteğin, birinciyi yapan kişiden geldiğini nereden bilecek?*"
>
> Öğrenciler genelde "IP adresinden" der. → **Karşı soru:** *"Aynı ağdaki 200 kişinin IP'si aynıysa?"*
>
> Sonra `Cookie` başlığını göster. **Söylenecek cümle:** "*Sunucu hatırlamıyor. Biz her seferinde ona kim olduğumuzu yeniden söylüyoruz. Bu satır Hafta 7'nin tamamı.*"

Yedek plan (internet yoksa): `H01-detay.md` §3'teki **hazır ham istek/yanıt metni** tahtaya veya slayta yansıtılır; aynı dört soru bunun üzerinden sorulur. DevTools gösterimi Hafta 2'ye devredilir.

Uygulama B — Sıfırdan Proje: İskeletin Turu

Süre: 14 dakika · **Blok:** B **Amaç:** Öğrencinin evde yapacağı ev görevini **bir kez baştan sona görmesi**; klasör yapısının soyut bir liste değil, gerçek bir çıktı olduğunu göstermek.

Adım 1 — Projeyi oluştur

```
cd ~/ders/bos
dotnet new mvc -n IlkProje
cd IlkProje
```

Beklenen çıktı:

```
The template "ASP.NET Core Web App (Model-View-Controller)" was created successfully.
```

> **Sınıfa sor:** *"Bu komut kaç dosya üretti sizce?"* → `ls -R` veya `dir /s` ile göster. Şaşırtıcı derecede çoktur; **bu, şablonun işini yaptığının kanıtıdır.**

Adım 2 — Klasörleri gez

```
ls
```

Sırayla aç ve **tek cümleyle** tanıt: `Controllers/`, `Views/`, `Models/`, `wwwroot/`, `Program.cs`, `appsettings.json`, `IlkProje.csproj`

> **Sınıfa sor:** *"Bu klasörlerden hangisinin içindekiler tarayıcıdan doğrudan indirilebilir?"* → `wwwroot`.
> **Devam sorusu:** *"O zaman veritabanı parolanızı hangi dosyaya koymazsınız?"*

⚠️ **Sık hata:** `wwwroot` ile `Views` karıştırılır. **Ayrım cümlesi:** "`Views` *sunucuda çalışır ve HTML üretir;* `wwwroot` *olduğu gibi indirilir.*"

Adım 3 — `Program.cs`'i aç ve dört evreyi işaretle

Dosyayı aç, ekranda **dört evreyi elle işaretle** (imleçle seç veya yorum satırı ekle):

```
var builder = WebApplication.CreateBuilder(args);      // 1. kurucu

builder.Services.AddControllersWithViews();            // 2. servis kaydı

var app = builder.Build();                             // 3. inşa

app.UseHttpsRedirection();                             // 4. pipeline
app.UseRouting();
app.UseAuthorization();
app.MapStaticAssets();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

> **Sınıfa sor:** *"Bu dosyada kaç satır gerçek iş yapıyor, kaç satır ayar?"* **Söylenecek cümle:** *"Bütün uygulamanız bu dosyadan başlıyor. Dönem boyunca buraya satır ekleyeceğiz — Hafta 7'de servis, Hafta 8'de veritabanı, Hafta 11'de kimlik."*

Adım 4 — Çalıştır

```
dotnet run
```

Beklenen çıktı:

```
Now listening on: https://localhost:7xxx
Application started. Press Ctrl+C to shut down.
```

Adresi tarayıcıya yapıştır → **Welcome** sayfası.

> **Sınıfa sor:** *"Adres çubuğuna hiçbir sayfa adı yazmadım. Sunucu hangi kodu çalıştıracağını nereden bildi?"*
>
> Cevabı **verme.** *"Bu, haftaya Hafta 2'nin ilk sorusu."* → Ders notundaki "Düşünerek gelin" sorusuyla aynıdır; köprü burada kurulur.

⚠️ **Sık hata (öğrenciler evde yaşayacak):** `Address already in use`. Sebebi: önceki `dotnet run` hâlâ açık. Çözüm: **Ctrl+C**. Bunu **canlı göster** — ikinci bir terminalde `dotnet run` çalıştırıp hatayı çıkar.

Yedek plan (ağ/paket sorunu): Hazırdaki `~/ders/IlkProje` klasörüne geç ve doğrudan `dotnet run` ile devam et. Adım 1 atlanır, Adım 2-4 aynen uygulanır.

Uygulama C — Yolculuğu Canlı İzlemek

Süre: 12 dakika · **Blok:** B **Amaç:** Ders notundaki "beş duraklı zincir" şemasının uydurma olmadığını göstermek; öğrencinin kendi gözüyle **sunucunun döndürdüğü ham yanıtı** görmesi.

Adım 1 — Yanıt başlıklarını oku

```
DevTools → Network → ilk "document" isteği → Headers → Response Headers
```

> **Sınıfa sor (tahmin ettir, sonra göster):**
>
> 1. "*Bu yanıtı hangi program üretti?*" → `Server` başlığı (varsa) veya ters vekil gizler
> 2. "*Bu sayfa önbelleklenecek mi?*" → `Cache-Control`
> 3. "*Sunucu içeriğin tipini nasıl söylüyor?*" → `Content-Type`

Vurgulanacak nokta: "*Bu başlıkların hepsini **siz** belirleyeceksiniz. Dönem boyunca bu satırları yazan taraf olacağız.*"

Adım 2 — Önbelleği canlı göster

```
Network sekmesi → "Disable cache" KAPALI → sayfayı iki kez yenile
Size sütununa bak → ikinci yüklemede "(memory cache)" / "(disk cache)"
Sonra "Disable cache" AÇ → yenile → boyutlar geri geldi
```

> **Söylenecek cümle:** "*Kodunuzu değiştirdiğinizde sayfanın değişmemesinin en sık sebebi bu. Dönem boyunca en az bir kez başınıza gelecek — o zaman buraya bakın.*"

⚠️ **Sık hata:** Öğrenci kodu değiştirir, sayfayı yeniler, değişiklik görünmez ve kodu bozduğunu sanır. Çözüm: <kbd>Ctrl</kbd>+<kbd>F5</kbd>.

Adım 3 — Zincirin bizim olmayan kısmını göster

```
Aynı istekte → Timing sekmesi
```

Beklenen çıktı: `DNS Lookup`, `Initial connection`, `SSL`, `Waiting (TTFB)`, `Content Download`

> **Sınıfa sor:** "*Bu sürelerden hangisi bizim kodumuzun suçu?*" → Yalnızca **Waiting (TTFB)**'nin bir kısmı. DNS, bağlantı ve SSL bizim elimizde değil.
>
> **Bağla:** "*İşte ders notundaki şemanın soldaki iki durağı bunlar. Bilgisayar Ağları dersi.*"

Yedek plan (internet yoksa): Etkileşimli sayfadaki **adım adım yolculuk** bileşeni (bölüm *Yolculuk — adım adım*) aynı dokuz adımı tıklamayla gezdirir; `H01-detay.md` §3'teki hazır ham istek/yanıt metni de kullanılabilir.

Uygulama D — Durum Kodu Kartları (grup etkinliği)

Süre: Kalan zamana göre 0-8 dakika · **Blok:** A veya C sonu **Amaç:** Durum kodu tablosunu ezber olmaktan çıkarıp karar hâline getirmek.

Sekiz senaryo okunur, sınıf **hep birlikte** kodu söyler:

#	Senaryo	Beklenen	Tuzak
1	Ana sayfa başarıyla açıldı	`200`	—
2	Yeni kullanıcı kaydedildi	`201`	`200` denir; "oluşturuldu" farkı vurgulanır
3	Kayıt silindi, dönecek içerik yok	`204`	`200` denir
4	Var olmayan bir ürün sayfası istendi	`404`	—
5	Giriş yapmadan "Profilim"e gidildi	`401`	`403` denir → *"sistem seni tanımıyor"*
6	Normal kullanıcı yönetim paneline gitti	`403`	`401` denir → *"tanıyor ama izin yok"*
7	Sunucuda `NullReferenceException` oluştu	`500`	`400` denir → **kimin hatası?**
8	Form gönderildi, zorunlu alan boş	`400`	`500` denir → **kimin hatası?**

> 7 ve 8 bilinçli olarak yan yanadır: **4xx = istemcinin hatası, 5xx = bizim hatamız.** Ders notundaki kuralın sınıf önünde test edilmesidir.

Yedek plan: Etkileşimli sayfadaki *Durum kodu kartları* bölümü aynı sekiz kartı tıklamalı olarak içerir; kâğıt gerekmez.

🏠 Ders Sonrası Öğrenci Görevi

`00-kurulum-rehberi.md` izlenerek:

1. .NET 10 SDK + bir IDE kurulur
2. `dotnet new mvc -n IlkProje` ile proje oluşturulur ve `dotnet run` ile çalıştırılır
3. **Teslim:** `dotnet --version` çıktısının ve tarayıcıda açılan sayfanın ekran görüntüsü

İsteğe bağlı (yapana artı): Kendi projesinde bir sayfayı iki kez yükleyip DevTools → Network'te **önbellekten gelen** isteği yakalar ve ekran görüntüsünü gönderir (Uygulama C Adım 2'nin tekrarı).

> Bu görev **puan için değil, "kurulum tamam" onayı içindir.** Hafta 2'nin ev görevi bunun üzerine kurulur; yapmayan öğrenci geride kalır.

✅ Hocaya Kontrol Listesi

Ders öncesi

- [ ] Terminal yazı tipi ≥ 18 punto, tema açık renk
- [ ] `~/ders/IlkProje` (hazır, restore edilmiş) ve `~/ders/bos` klasörleri yerinde
- [ ] `dotnet --version` → 10.x
- [ ] Projeksiyon çözünürlüğü ayarlı; kod okunuyor mu diye **arka sıradan** bakıldı
- [ ] Tarayıcı yakınlaştırma en az %125

Ders içi

- [ ] Uygulama C Adım 2 ve 3'te **önce tahmin ettirildi, sonra çalıştırıldı** (tersi yapılırsa etkinin tamamı kaybolur)
- [ ] Uygulama B Adım 4'ün "sunucu nereden bildi" sorusunun **cevabı verilmedi**
- [ ] Değiştirilen kod satırları **geri alındı** (sonraki derste aynı proje kullanılacak)

Ders sonrası

- [ ] Ev görevi duyuruldu ve teslim biçimi söylendi
- [ ] Kurulum yapamayacağını söyleyen öğrenci **not edildi** (bkz. plan §12.17 — çözüm dönemin ilk iki haftasında bulunmalı)

⚠️ Yedek Planların Özeti

Sorun	Ne yapılır
İnternet yok	Uygulama A → `H01-detay.md` §3'teki hazır ham metin. Uygulama B → hazır proje klasörü
`dotnet new` paket indiremiyor	Hazırdaki `~/ders/IlkProje` ile devam
Projeksiyon kodu okunamıyor	Etkileşimli sayfadaki kod bölümleri (yazı boyutu 19px taban) kullanılır
Hiçbir şey çalışmıyor	Etkileşimli sayfadaki **pipeline canlandırması** + durum kodu kartları; üç uygulama da tıklamayla yapılır
Süre taştı	Uygulama D atlanır → Uygulama C Adım 3 atlanır → **Uygulama C Adım 2 asla atlanmaz**